



数据结构

- 信息标记类型

```
struct Tag{
    Tag(){}
    bool operator==(const Tag &x)const{return 1;}
    Tag operator+(const Tag &x){}
};
struct Info{
    Info(){}
    Info operator+(const Info &x)const{}
    Info operator+(const Tag &x)const{}
};
```

- *Trie*01 01字典树
- 支持插入一个整数、删除一个整数、查询与给定整数异或的最大结果。
- 通常用于处理最大异或对等问题。

```

struct Trie01{
    struct Node{
        int ch[2],val;
    }t[maxn];
#define ch(p) t[p].ch
#define val(p) t[p].val
    int idx=1;
    void Add(int x,int k){
        int p=1;
        for(int i=30;i>=0;--i){
            bool bi=(x>>i)&1;
            if(!ch(p)[bi]) ch(p)[bi]=++idx;
            p=ch(p)[bi],val(p)+=k;
        }
    }
    int Query(int x){
        int p=1,ans=0;
        for(int i=30;i>=0;--i){
            bool bi=(x>>i)&1;
            if(val(ch(p)[bi^1])>=1)
                ans+=(1<<i),p=ch(p)[bi^1];
            else p=ch(p)[bi];
        }
        return ans;
    }
    void Init(){
        for(int i=1;i<=idx;++i)
            ch(i)[0]=ch(i)[1]=0,val(i)=0;
        idx=1;
    }
};

```

- *Dkt* 笛卡尔树
- 根据数组 a 构建小根笛卡尔树, `ls/rs` 为左右孩子。
- *Build* 后可通过 `ls/rs` 遍历笛卡尔树。

```

struct Dkt{
    int st[maxn],top;
    int a[maxn],ls[maxn],rs[maxn];
    void Build(){
        for(int i=1;i<=n;++i){
            int cur=top;
            while(cur&&a[st[cur]]>a[i])
                --cur;
            if(cur) rs[st[cur]]=i;
            if(cur<top) ls[i]=st[cur+1];
            top=cur;
            st[++top]=i;
        }
    }
};

```

- *STtable* 稀疏表 (*SparseTable*)
- 用于离线查询区间最大值(或其他可重复贡献值)。
- 预处理 $O(n\log n)$, 单次查询 $O(1)$ 。

```

struct STtable{
    ll mxa[maxn][25];
    ll Log2[maxn];
    ll GetMax(ll l,ll r){
        ll dif=Log2[r-l+1];
        return max(mxa[l][dif],mxa[r-(1<<dif)+1][dif]);
    }
    void Init(){
        for(ll k=1;k<=20;++k) for(ll i=1;i<=n;++i)
            mxa[i][k]=max(mxa[i+(1<<(k-1))][k-1],mxa[i][k-1]);
        Log2[1]=0;
        for(int i=2;i<=n;++i)
            Log2[i]=Log2[i/2]+1;
    }
};

```

- *FHQTreap2* 可分裂结点的 *FHQTreap* (示例, 未完整实现)
- 支持 *SplitNode*: 按长度分裂结点, 常用于文本编辑器等需要维护块长的结构。

- 核心操作: *SplitS* 支持按 *siz* 分裂, *Merge* 合并, *Del* 回收结点。
- 注意: 部分函数体未完成, 需要根据实际使用补全 *PushUp* 等。

```

struct FHQTreap2{
    struct Node{
        int ls,rs;
        ll val,pri,len,siz;
    }t[maxn<<1];
    int rt,idx;
    mt19937 mt;
    stack<int>s;
#define ls(p) t[p].ls
#define rs(p) t[p].rs
#define val(p) t[p].val
#define mx(p) t[p].mx
#define tag(p) t[p].tag
#define pri(p) t[p].pri
#define len(p) t[p].len
#define siz(p) t[p].siz
    void PushUp(int p);
    void AddTag(int p,ll k);
    void PushDown(int p);
    int NewNode(ll val,int len){
        int id;
        if(s.size()) id=s.top(),s.pop();
        else id=++idx;
        t[id]={0,0,val,(int)mt(),len,len};
        return id;
    }
    pair<int,int> SplitNode(int p,int sz){
        int p2=NewNode(val(p),len(p)-sz);
        len(p)=sz;
        rs(p2)=rs(p),rs(p)=0;
        PushUp(p),PushUp(p2);
        return {p,p2};
    }
    void SplitS(int p,int sz,int &l,int &r){
        if(!p) return l=r=0,void();
        PushDown(p);
        if(siz(ls(p))>=sz){
            r=p;
            SplitS(ls(p),sz,l,ls(p));
        }
    }

```

```

    }else if(siz(ls(p))+len(p)<=sz){
        l=p;
        SplitS(rs(p),sz-siz(ls(p))-len(p),rs(p),r);
    }else{
        auto [p1,p2]=SplitNode(p,sz-siz(ls(p)));
        l=p1,r=p2;
    }PushUp(p);
}
int Merge(int l,int r){
    if(!l||!r) return l|r;
    if(pri(l)>pri(r)){
        PushDown(l);
        rs(l)=Merge(rs(l),r);
        PushUp(l);
        return l;
    }else{
        PushDown(r);
        ls(r)=Merge(l,ls(r));
        PushUp(r);
        return r;
    }
}
queue<int>q;
void Del(int p){
    q.push(p);
    while(!q.empty()){
        p=q.front();q.pop();
        if(!p) continue;
        s.push(p);
        q.push(ls(p));
        q.push(rs(p));
    }
}
void Fun(int l,int r){
    int L,p,R;
    SplitS(rt,l-1,L,p);
    SplitS(p,r-1+1,p,R);
    //
    rt=Merge(L,Merge(p,R));
}

```

```
void Init(){  
    }  
};
```

- *FHQTreap* 维护信息与标记的 *FHQTreap*
- 支持区间加、区间查询等操作。
- 结点存储 *Info* 与 *Tag*, 需与具体题目定义的 *Info/Tag* 配合。

```

struct FHQTreap{
    struct Node{
        int ls,rs,pri,siz;
        Info val,sum;
        Tag tag;
    }t[maxn<<1];
    int rt,idx;
    mt19937 mt;
#define ls(p) t[p].ls
#define rs(p) t[p].rs
#define val(p) t[p].val
#define sum(p) t[p].sum
#define tag(p) t[p].tag
#define pri(p) t[p].pri
#define siz(p) t[p].siz
    void PushUp(int p){
        sum(p)=sum(ls(p))+val(p)+sum(rs(p));
        siz(p)=siz(ls(p))+siz(rs(p))+1;
    }
    void AddTag(int p,Tag k){
        val(p)=val(p)+k,sum(p)=sum(p)+k;
        tag(p)=tag(p)+k;
    }
    void PushDown(int p){
        if(tag(p)==Tag{}) return;
        AddTag(ls(p),tag(p)),AddTag(rs(p),tag(p));
        tag(p)={};
    }
    int NewNode(Info val){
        t[++idx]={0,0,int(mt()),1,val,val,{} };
        return idx;
    }
    void SplitS(int p,int sz,int &l,int &r){
        if(!p) return l=r=0,void();
        PushDown(p);
        if(siz(ls(p))+1<=sz){
            l=p;
            SplitS(rs(p),sz-siz(ls(p))-1,rs(p),r);
        }else{

```



```

        r=p;
        SplitS(ls(p),sz,l,ls(p));
    }PushUp(p);
}
int Merge(int l,int r){
    if(!l||!r) return l|r;
    if(pri(l)>pri(r)){
        PushDown(l);
        rs(l)=Merge(rs(l),r);
        PushUp(l);
        return l;
    }else{
        PushDown(r);
        ls(r)=Merge(l,ls(r));
        PushUp(r);
        return r;
    }
}
Info Query(int l,int r){
    int L,p,R;
    SplitS(rt,l-1,L,p);
    SplitS(p,r-1+1,p,R);
    Info ans=sum(p);
    rt=Merge(L,Merge(p,R));
    return ans;
}
void Add(int l,int r,Tag k){
    int L,p,R;
    SplitS(rt,l-1,L,p);
    SplitS(p,r-1+1,p,R);
    AddTag(p,k);
    rt=Merge(L,Merge(p,R));
}
void Init(){
    sum(0)=val(0)={};
    tag(0)={};
}
};

```

- *SegTree* 线段树 (单点修改、区间加、区间查询)

```

struct SegTree{
    struct Node{
        Info sum;
        Tag tag;
    }t[maxn<<2];
#define ls(p) (p<<1)
#define rs(p) (p<<1|1)
#define sum(p) t[p].sum
#define tag(p) t[p].tag
    void AddTag(int p,Tag k){
        tag(p)=tag(p)+k;
        sum(p)=sum(p)+k;
    }
    void PushDown(int p){
        if(tag(p)==Tag()) return;
        AddTag(ls(p),tag(p));
        AddTag(rs(p),tag(p));
        tag(p)={};
    }
    void PushUp(int p){
        sum(p)=sum(ls(p))+sum(rs(p));
    }
    void Add(int p,int l,int r,int L,int R,Tag k){
        if(L<=l&&R>=r) return AddTag(p,k);
        if(l>R||r<L) return;
        PushDown(p);
        int mid=l+r>>1;
        Add(ls(p),l,mid,L,R,k);
        Add(rs(p),mid+1,r,L,R,k);
        PushUp(p);
    }
    void Change(int p,int l,int r,int x,Info k){
        if(l==r) return sum(p)=k,void();
        PushDown(p);
        int mid=l+r>>1;
        if(x<=mid) Change(ls(p),l,mid,x,k);
        else Change(rs(p),mid+1,r,x,k);
        PushUp(p);
    }
}

```

```

Info Query(int p,int l,int r,int L,int R){
    if(L<=l&&R>=r) return sum(p);
    if(l>R||r<L) return {};
    PushDown(p);
    int mid=l+r>>1;
    return Query(ls(p),l,mid,L,R)+Query(rs(p),mid+1,r,L,R);
}
void Add(int l,int r,Tag k){
    Add(1,1,n,l,r,k);
}
Info Query(int l,int r){
    return Query(1,1,n,l,r);
}
void Build(int p,int l,int r){
    sum(p)={},tag(p)={};
    if(l==r) return sum(p)={},void();
    int mid=l+r>>1;
    Build(ls(p),l,mid);
    Build(rs(p),mid+1,r);
    PushUp(p);
}
};

```

- *DSegTree* 动态开点线段树（单点修改，前缀/区间查询）
- 多个线段树可以通过 *root* 数组索引，用于树套树或点分树。

```

struct DSegTree{
    int root[maxn],idx;
    struct Node{
        int ls,rs,sum;
    }t[maxn<<5];
#define ls(p) t[p].ls
#define rs(p) t[p].rs
#define sum(p) t[p].sum
    void PushUp(int p){
        sum(p)=sum(ls(p))+sum(rs(p));
    }
    void Change(int &p,int l,int r,int x,int k){
        if(!p) p=++idx;
        if(l==r) return sum(p)+=k,void();
        int mid=l+r>>1;
        if(x<=mid) Change(ls(p),l,mid,x,k);
        else Change(rs(p),mid+1,r,x,k);
        PushUp(p);
    }
    int Query(int p,int l,int r,int L,int R){
        if(l>R||r<L||!p) return 0;
        if(L<=l&&R>=r) return sum(p);
        int mid=l+r>>1;
        return Query(ls(p),l,mid,L,R)+Query(rs(p),mid+1,r,L,R);
    }
    void Change(int u,int x,int k){
        Change(root[u],0,n,x,k);
    }
    int Query(int u,int x){
        return Query(root[u],0,n,0,x);
    }
};

```

- *CSegTree* 可持久化线段树 (主席树)
- 支持区间第 k 小查询。

```

struct CSegTree{
    struct node{
        int ls,rs,sum;
    }t[maxn<<4];
#define ls(p) t[p].ls
#define rs(p) t[p].rs
#define sum(p) t[p].sum
    int root[maxn],idx;
    int CopyNode(int p){
        t[++idx]=t[p];
        return idx;
    }
    void PushUp(int p){
        sum(p)=sum(ls(p))+sum(rs(p));
    }
    void Change(int &p,int l,int r,int k){
        p=CopyNode(p);
        if(l==r) return sum(p)++,void();
        int mid=l+r>>1;
        if(k<=mid) Change(ls(p),l,mid,k);
        else Change(rs(p),mid+1,r,k);
        PushUp(p);
    }
    int Query(int u,int v,int l,int r,int k){
        if(l==r) return l;
        int s=sum(ls(v))-sum(ls(u)),mid=l+r>>1;
        if(s>=k) return Query(ls(u),ls(v),l,mid,k);
        return Query(rs(u),rs(v),mid+1,r,k-s);
    }
    int Query(int l,int r,int k){
        return Query(root[l-1],root[r],1,n,k);
    }
    void Init(){
        for(int i=1;i<=n;++i){
            root[i]=root[i-1];
            Change(root[i],1,n,a[i]);
        }
    }
};

```

- *BitTree2* 二维树状数组 (使用 *unordered map*, 动态开点)
- 支持单点加、二维前缀查询。

```
struct BitTree2{
    unordered_map<int,int>t[maxn];
#define lowbit(x) (x&(-x))
    void Add(int x,int y,int k){
        while(x<=n){
            for(int i=y;i<=n;i+=lowbit(i))
                t[x][i]+=k;
            x+=lowbit(x);
        }
    }
    int Query(int x,int y){
        int ans=0;
        while(x){
            for(int i=y;i;i-=lowbit(i))
                ans+=t[x][i];
            x-=lowbit(x);
        }
        return ans;
    }
};
```

- *LCT* 动态树 (*Link – CutTree*)
- 支持树的连边、断边、路径查询、路径加等操作。
- 结点维护 *Info* 信息与 *Tag* 标记。

```

struct LCT{
    struct Node{
        int ch[2],rev,fa;
        Info val,sum;
        Tag tag;
    }t[maxn];
#define ls(p) t[p].ch[0]
#define rs(p) t[p].ch[1]
#define ch(p) t[p].ch
#define val(p) t[p].val
#define sum(p) t[p].sum
#define rev(p) t[p].rev
#define tag(p) t[p].tag
#define fa(p) t[p].fa
    int idx,rt;
    void Rev(int p){
        swap(rs(p),ls(p));
        rev(p)^=1;
    }
    int NewNode(Info val){
        t[++idx]={0,0,0,0,val,val,{} };
        return idx;
    }
    void AddTag(int p,Tag k){
        sum(p)=sum(p)+k,val(p)=val(p)+k;
        tag(p)=tag(p)+k;
    }
    void PushUp(int p){
        sum(p)=sum(ls(p))+val(p)+sum(rs(p));
    }
    void PushDown(int p){
        if(rev(p)) Rev(ls(p)),Rev(rs(p)),rev(p)=0;
        if(tag(p)==Tag()) return;
        AddTag(ls(p),tag(p)),AddTag(rs(p),tag(p));
        tag(p)={ };
    }
    void PushAll(int p){
        if(!IsRoot(p)) PushAll(fa(p));
        PushDown(p);
    }

```

```

}
bool IsRoot(int p){
    return ls(fa(p))!=p&&rs(fa(p))!=p;
}
void Rotate(int p){
    int fp=fa(p),ffp=fa(fp),k=rs(fp)==p;
    if(!IsRoot(fp)){
        ch(ffp)[rs(ffp)==fp]=p;
    }fa(p)=ffp;
    ch(fp)[k]=ch(p)[k^1],fa(ch(p)[k^1])=fp;
    ch(p)[k^1]=fp,fa(fp)=p;
    PushUp(fp),PushUp(p);
}
void Splay(int p){
    PushAll(p);
    while(!IsRoot(p)){
        int fp=fa(p),ffp=fa(fp);
        if(!IsRoot(fp))
            Rotate((ls(fp)==p)^(ls(ffp)==fp)?p:fp);
        Rotate(p);
    }
}
void Access(int p){
    for(int sp=0;p;p=fa(p)){
        Splay(p);
        rs(p)=sp;
        PushUp(p);
    }
}
void MakeRoot(int p){
    Access(p);
    Splay(p);
    Rev(p);
}
int FindRoot(int p){
    Access(p);
    Splay(p);
    while(ls(p))
        PushDown(p),p=ls(p);
    Splay(p);
}

```



```

        return p;
    }
    void Link(int u,int v){
        MakeRoot(u);
        if(FindRoot(v)!=u)
            fa(u)=v;
    }
    void Cut(int u,int v){
        MakeRoot(u);
        Access(v);
        Splay(v);
        if(ls(v)==u&&!rs(u)&&fa(u)==v)
            ls(v)=fa(u)=0;
        PushUp(v);
    }
    Info Query(int u,int v){
        MakeRoot(u);
        Access(v);
        Splay(v);
        return sum(v);
    }
    void Add(int u,int v,Tag k){
        MakeRoot(u);
        Access(v);
        Splay(v);
        AddTag(v,k);
    }
    void Init(){
        val(0)=sum(0)={};
        tag(0)={};
    }
};

```

- *Tree* 轻重链剖分 (*Heavy – LightDecomposition*)
- 支持求 *LCA*、树上两点距离, 以及提供 *top* 等剖分信息用于链上操作。

```

struct Tree{
    vector<int>e[maxn];
    int dep[maxn],fa[maxn],son[maxn],siz[maxn],top[maxn];
    void Dfs1(int u,int f){
        dep[u]=dep[fa[u]=f]+(siz[u]=1);
        for(int v:e[u]) if(v!=f){
            Dfs1(v,u);
            siz[u]+=siz[v];
            if(siz[v]>siz[son[u]])
                son[u]=v;
        }
    }
    void Dfs2(int u,int tp){
        top[u]=tp;
        if(!son[u]) return;
        Dfs2(son[u],tp);
        for(int v:e[u]) if(v!=fa[u]&&v!=son[u])
            Dfs2(v,v);
    }
    int GetLCA(int u,int v){
        while(top[u]!=top[v]){
            if(dep[top[u]]<dep[top[v]])
                swap(u,v);
            u=fa[top[u]];
        }
        return dep[u]>dep[v]?v:u;
    }
    int GetDis(int u,int v){
        int lca=GetLCA(u,v);
        return dep[u]+dep[v]-2*dep[lca];
    }
    void Init(){
        Dfs1(1,0);
        Dfs2(1,1);
    }
};

```

- *TopTree* 树簇 (*TopTree*) 的实现 (用于动态 *DP* 等)
- 使用 *Compress* 与 *Rake* 操作合并信息, 支持修改点权并查询全局最大权独立集等。

- 结构包含 *Build* 构建全局树簇, *Change* 单点修改, *Query* 查答案。

```

namespace TopTree{
    const int maxn=1e6+10,INF=0x3f3f3f3f,mod=1e9+7;
    int n,m;
    int a[maxn];
    vector<int>e[maxn];
    int siz[maxn],son[maxn],fa[maxn];
    struct Info{
        int M[2][2];
        Info(int val=-INF){
            M[0][0]=M[0][1]=0;
            M[1][0]=val,M[1][1]=-INF;
        }
        int *operator[](const int x){
            return M[x];
        }
    };
    Info Compress(Info &u,Info &v){
        Info res;
        res[0][0]=max(v[0][0]+u[0][0],v[0][1]+u[1][0]);
        res[0][1]=max(v[0][0]+u[0][1],v[0][1]+u[1][1]);
        res[1][0]=max(v[1][0]+u[0][0],v[1][1]+u[1][0]);
        res[1][1]=max(v[1][0]+u[0][1],v[1][1]+u[1][1]);
        return res;
    }
    Info Rake(Info &u,Info &v){
        Info res;
        res[0][0]=u[0][0]+max(v[0][0],v[1][0]);
        res[0][1]=u[0][1]+max(v[0][1],v[1][1]);
        res[1][0]=u[1][0]+max(v[0][0],v[1][0]);
        res[1][1]=u[1][1]+max(v[0][1],v[1][1]);
        return res;
    }
    struct TopTree{
        struct Node{
            int ls,rs,fa;
            Info val;
            bool ty;
            int siz;
        }t[maxn];
    };
}

```

```

#define ls(p) t[p].ls
#define rs(p) t[p].rs
#define fa(p) t[p].fa
#define val(p) t[p].val
#define siz(p) t[p].siz
#define ty(p) t[p].ty
    int idx,rt;
    int NewNode(bool ty,int val=-INF){
        t[++idx]={0,0,0,{val},ty,1};
        return idx;
    }
    void PushUp(int p){
        siz(p)=siz(ls(p))+siz(rs(p));
        val(p)=(ty(p)?Rake:Compress)(val(ls(p)),val(rs(p)));
    }
    int Merge(int u,int v,bool ty){
        int p=NewNode(ty);
        fa(ls(p)=u)=fa(rs(p)=v)=p;
        PushUp(p);
        return p;
    }
    int SubBuild(vector<int>&C,int l,int r,bool ty){
        if(l==r) return C[l];
        int sum=0,now=0;
        for(int i=l;i<=r;++i)
            sum+=siz(C[i]);
        for(int i=l;i<r;++i) if((now+=siz(C[i]))*2>=sum||i==r-1)
            return Merge(SubBuild(C,l,i,ty),SubBuild(C,i+1,r,ty),ty);
    }
    int Build(int p){
        vector<int>H;
        if(p<=n) H.push_back(p);
        for(int u=p;son[u];u=son[u]){
            vector<int>L;
            for(int v:e[u]) if(v^fa[u]&&v^son[u])
                L.push_back(Build(v));
            if(!L.size()) H.push_back(son[u]);
            else H.push_back(Merge(son[u],SubBuild(L,0,L.size()-1,1),1));
        }
        return SubBuild(H,0,H.size()-1,0);
    }

```

```

    }
    void Change(int u,int k){
        val(u)={k};
        while(u=fa(u)) PushUp(u);
    }
    int Query(){
        return max(val(rt)[0][0],val(rt)[1][0]);
    }
    void Init(){
        for(int i=1;i<=n;++i)
            NewNode(0,a[i]);
        rt=Build(n+1);
    }
}t;
void Dfs(int u,int p){
    siz[u]=1,fa[u]=p;
    for(int v:e[u]) if(v^p)
        Dfs(v,u),siz[u]+=siz[v],siz[v]>siz[son[u]]?son[u]=v:0;
}
void solve(){
    int u,w;
    while(m--){
        cin>>u>>w;
        t.Change(u,w);
        cout<<t.Query()<<'\n';
    }
}
void init(){
    cin>>n>>m;
    for(int i=1;i<=n;++i)
        cin>>a[i];
    int u,v;
    for(int i=1;i<n;++i)
        cin>>u>>v,e[u].push_back(v),e[v].push_back(u);
    e[n+1].push_back(1),e[1].push_back(n+1);
    Dfs(n+1,0);
    t.Init();
}
}

```

- *NodeDivide* 点分治 (*CentroidDecomposition*)
- 用于解决树上路径统计问题, 此处统计距离 $\leq m$ 的点队数量。
- *BIT* 用于维护当前重心子树到重心的距离频次。

```

namespace NodeDivide{
    vector<pair<int,int>>e[maxn];
    int rt;
    int siz[maxn],mxp[maxn];
    bool vis[maxn];
    void GetRT(int u,int fa,int tot){
        siz[u]=1,mxp[u]=0;
        for(auto[v,w]:e[u])if(v!=fa&&!vis[v])
            GetRT(v,u,tot),siz[u]+=siz[v],mxp[u]=max(mxp[u],siz[v]);
        mxp[u]=max(mxp[u],tot-siz[u]);
        if(mxp[u]<mxp[rt]) rt=u;
    }
    struct BIT{
        int t[int(2e4)+100];
        void Add(int x,int k){
            x++;
            while(x<=m+10)
                t[x]+=k,x+=lowbit(x);
        }
        int Query(int x){
            x++;
            if(x>m+10) x=m+10;
            int ans=0;
            while(x)
                ans+=t[x],x-=lowbit(x);
            return ans;
        }
    }t;
    int dis[maxn];
    vector<int>d,st;
    void Dfs(int u,int fa){
        d.push_back(dis[u]);
        for(auto[v,w]:e[u]) if(v!=fa&&!vis[v])
            dis[v]=dis[u]+w,Dfs(v,u);
    }
    ll ans=0;
    void Calc(int u){
        for(auto[v,w]:e[u])if(!vis[v]){
            dis[v]=w;

```



```

        d.clear();
        Dfs(v,u);
        for(int k:d) if(k<=m)
            ans+=t.Query(m-k);
        for(int k:d) if(k<=m)
            t.Add(k,1),st.push_back(k);
    }
    for(int k:st) t.Add(k,-1);
    st.clear();
}

void Divide(int u){
    Calc(u);
    vis[u]=1;
    for(auto[v,w]:e[u]) if(!vis[v]){
        rt=0;
        GetRT(v,u,siz[v]);
        Divide(rt);
    }
}

void solve(){
    GetRT(1,0,n);
    Divide(rt);
    cout<<ans<<'\n';
}

void init(){
    cin>>n;
    int u,v,w;
    for(int i=1;i<n;++i)
        cin>>u>>v>>w,e[u].push_back({v,w}),e[v].push_back({u,w});
    cin>>m;
    t.Add(0,1);
    mxp[0]=INF;
}
}

```

- *CenTree* 点分树 (*CentroidTree*) + 动态开点线段树
- 支持动态修改点权, 查询与点 u 距离 $\leq k$ 的点的权值和。
- 点分树 fa 维护父子关系, lca 求原树中距离, t/f 是前缀距离权值线段树用于容斥。

```

namespace CenTree{
    int a[maxn];
    vector<int>e[maxn];
    struct CenTree{
        int siz[maxn],mxp[maxn],cen,fa[maxn];
        bitset<maxn>vis;
        Tree lca;
        DSegTree t,f;
        int w[maxn];
        void GetCen(int u,int fa,int tot){
            siz[u]=1,mxp[u]=0;
            for(int v:e[u]) if(v!=fa&&!vis[v])
                GetCen(v,u,tot),siz[u]+=siz[v],mxp[u]=max(mxp[u],siz[v]);
            mxp[u]=max(mxp[u],tot-siz[u]);
            if(!cen||mxp[u]<mxp[cen]) cen=u;
        }
        void Build(int u,int tot){
            vis[u]=1;
            for(int v:e[u])if(!vis[v]){
                cen=0;
                tot=siz[v];
                GetCen(v,u,tot);
                fa[cen]=u;
                Build(cen,tot);
            }
        }
        void Change(int u,ll k){
            int now=u;
            while(now){
                t.Change(now,lca.GetDis(u,now),k-w[u]);
                if(fa[now]) f.Change(now,lca.GetDis(u,fa[now]),k-w[u]);
                now=fa[now];
            }
            w[u]=k;
        }
        ll Query(int u,int k){
            ll ans=0,now=u,pre=0;
            while(now){
                int dis=lca.GetDis(now,u);

```

```

        if(dis<=k){
            ans+=t.Query(now,k-dis);
            if(pre) ans-=f.Query(pre,k-dis);
        }
        pre=now,now=fa[now];
    }
    return ans;
}

void Init(){
    lca.Init();
    GetCen(1,0,n);
    Build(cen,n);
    for(int i=1;i<=n;++i)
        Change(i,a[i]);
}

}t;

void solve(){
    ll lastans=0,op,x,k;
    while(m--){
        cin>>op>>x>>k;
        x^=lastans,k^=lastans;
        if(op) t.Change(x,k);
        else cout<<(lastans=t.Query(x,k))<<'\\n';
    }
}

void init(){
    cin>>n>>m;
    for(int i=1;i<=n;++i)
        cin>>a[i];
    int u,v;
    for(int i=1;i<n;++i)
        cin>>u>>v,e[u].push_back(v),e[v].push_back(u);
    t.Init();
}

}

```

- 带修莫队，区间数颜色

```

namespace C_Mo{
    int Blen;
    struct Query{
        int l,r,t,id;
        bool operator<(Query x){
            if(t/Blen!=x.t/Blen) return t>x.t;
            if(l/Blen!=x.l/Blen) return (t/Blen&1)? l<x.l:l>x.l;
            return (l/Blen&1)?r<x.r:r>x.r;
        }
    }q[maxn];
    struct Change{
        int pos,pre,now;
    }c[maxn];
    int l=1,r=0,t;
    int a[maxn],m1,m2,ans[maxn];
    int cnt[maxn],res;

    void Add(int u){
        if(!(cnt[u]++)) res++;
    }
    void Del(int u){
        if(!(--cnt[u])) res--;
    }
    void Cha(int u,bool i){
        auto [pos,pre,now]=c[u];
        if(i){
            if(pos>=l&&pos<=r)
                Del(pre),Add(now);
            a[pos]=now;
        }else{
            if(pos>=l&&pos<=r)
                Del(now),Add(pre);
            a[pos]=pre;
        }
    }

    void solve(){
        t=m2;
        for(int i=1;i<=m1;++i){

```

```

        while(t<q[i].t) Cha(++t,1);
        while(t>q[i].t) Cha(t--,0);
        while(r<q[i].r) Add(a[++r]);
        while(l>q[i].l) Add(a[--l]);
        while(r>q[i].r) Del(a[r--]);
        while(l<q[i].l) Del(a[l++]);
        ans[q[i].id]=res;
    }
    for(int i=1;i<=m1;++i)
        cout<<ans[i]<<'\\n';
}

void init(){
    cin>>n>>m;
    Blen=pow(n,0.7);
    for(int i=1;i<=n;++i) cin>>a[i];
    char ch;
    int u,v;
    while(m--){
        cin>>ch>>u>>v;
        if(ch=='Q') q[++m1]={u,v,m2,m1};
        else c[++m2]={u,a[u],v},a[u]=v;
    }
    sort(q+1,q+1+m1);
}
}

```

- 莫队 + 树状数组二分，区间内最小没有出现过的自然数

```

namespace Mo{
    struct BIT{
        int t[maxn];
        #define lowbit(x) (x&(-x))
        #define LOG 17
        void Add(int x,int k){
            while(x<=maxn ){
                t[x]+=k;
                x+=lowbit(x);
            }
        }
        int Query(){
            int pos=0,nx;
            for(int i=LOG;i>=0;--i){
                nx=pos+(1<<i);
                if(t[nx]==(1<<i))
                    pos=nx;
            }
            return pos+1;
        }
    }t;
    int Blen;
    struct Que{
        int l,r,id;
        bool operator<(Que x){
            if(l/Blen!=x.l/Blen) return l<x.l;
            return (r<x.r);
        }
    }q[maxn];
    int cnt[maxn],ans[maxn];
    void Add(int x){
        if(!(cnt[x]++)) t.Add(x,1);
    }
    void Del(int x){
        if(!(--cnt[x])) t.Add(x,-1);
    }
    void solve(){
        int l=1,r=0;
        for(int i=1;i<=m;++i){

```

```

        while(r<q[i].r) Add(a[++r]);
        while(l>q[i].l) Add(a[--l]);
        while(r>q[i].r) Del(a[r--]);
        while(l<q[i].l) Del(a[l++]);
        ans[q[i].id]=t.Query()-1;
    }
    for(int i=1;i<=m;++i)
        printf("%d\n",ans[i]);
}
void init(){
    cin>>n>>m;
    Blen=sqrt(n);
    for(int i=1;i<=n;++i)
        cin>>a[i],a[i]++;
    for(int i=1;i<=m;++i)
        cin>>q[i].l>>q[i].r,q[i].id=i;
    sort(q+1,q+1+m);
}
}

```

- 分块，区间加，区间询问>=C

```

struct Block1{
    int a[maxn],c[maxn],tag[maxn];
    int Bnum,Blen;
    int b[maxn];
    void PushDown(int bi){
        if(!tag[bi]) return;
        for(int i=b[bi-1]+1;i<=b[bi];++i)
            a[i]+=tag[bi],c[i]+=tag[bi];
        tag[bi]=0;
    }
    // 重构
    void Make(int bi){
        PushDown(bi);
        for(int i=b[bi-1]+1;i<=b[bi];++i)
            c[i]=a[i];
        sort(c+b[bi-1]+1,c+b[bi]+1);
    }
    // 在 c 中查找 >= k 的元素个数
    int Find(int bi,int k){
        int l=b[bi-1]+1,r=b[bi];
        if(c[r]<k) return 0;
        while(l<r){
            int mid=l+r>>1;
            if(c[mid]>=k) r=mid;
            else l=mid+1;
        }
        return b[bi]-r+1;
    }
    int Query(int l,int r,int k){
        int ql=(l-1)/Blen+1,qr=min((r-1)/Blen+1,Bnum),ans=0;
        PushDown(ql);
        for(int i=l;i<=min(b[ql],r);++i)
            ans+=a[i]>=k;
        if(ql!=qr){
            PushDown(qr);
            for(int i=b[qr-1]+1;i<=r;++i)
                ans+=a[i]>=k;
        }
        for(int i=ql+1;i<qr;++i)

```



```

        ans+=Find(i,k-tag[i]);
    return ans;
}
void Add(int l,int r,int k){
    int ql=(l-1)/Blen+1,qr=min((r-1)/Blen+1,Bnum);
    for(int i=l;i<=min(b[ql],r);++i)
        a[i]+=k;
    Make(ql);
    if(ql!=qr){
        for(int i=b[qr-1]+1;i<=r;++i)
            a[i]+=k;
        Make(qr);
    }
    for(int i=ql+1;i<qr;++i)
        tag[i]+=k;
}
void init(){
    Blen=sqrt(n);
    Bnum=n/Blen+(Blen*Blen<n);
    for(int i=1;i<Bnum;++i)
        b[i]=i*Blen;
    b[Bnum]=n;
    for(int i=1;i<=Bnum;++i)
        Make(i);
}
};

```

图论

- 链式前向星

```

#define ForE(u) for(int i=head[u],v=e[i].v,w=e[i].w;i;i=e[i].nx,v=e[i].v,w=e[i].w)

struct Edge{
    ll u,v,w,nx;
    Edge(){}
    Edge(int U,int V,int W,int Nx):u(U),v(V),w(W),nx(Nx){}
    bool operator<(const Edge &x)const{return w<x.w;}
}e[maxn*4];

int head[maxn],midx=1;

void AddEdge(int u,int v,int w=1){
    if(u==v) return;
    e[++midx]=Edge(u,v,w,head[u]);
    head[u]=midx;
}

```

- *SPFA*

```

bool inq[maxn];
int mcnt[maxn],h[maxn];
bool Spfa(int s){
    queue<int>q;
    for(int i=0;i<=n;++i) h[i]=INF;
    q.push(s);
    h[s]=0;
    inq[s]=1;
    while (q.size()){
        int u =q.front();
        q.pop();
        inq[u]=0;
        ForE(u){
            if(h[v]>h[u]+w){
                h[v]=h[u]+w;
                mcnt[v]=mcnt[u]+1;
                if(mcnt[v]>n) return 0;
                if(!inq[v]) q.push(v),inq[v]=1;
            }
        }
    }
    return 1;
}

```

- *Dijkstra*

```

int d[maxn];
bool vis[maxn];
void Dijstra(int s){
    priority_queue<pair<int,int>>q;
    memset(d,0x3f,sizeof(d));
    memset(vis,0,sizeof(vis));
    q.push({0,s});
    d[s]=0;
    while(!q.empty()){
        auto u=q.top().second;q.pop();
        if(vis[u]) continue;
        vis[u]=1;
        ForE(u){
            if(d[v]>d[u]+w){
                d[v]=d[u]+w;
                if(!vis[v]) q.push({-d[v],v});
            }
        }
    }
}

```

- *Kruskal*

```

int fa[maxn];
int Findfa(int x){
    if(fa[x]==x) return x;
    return fa[x]=Findfa(fa[x]);
}
void Merge(int x,int y){
    fa[Findfa(x)]=Findfa(y);
}
int Kruskal(){
    for(int i=1;i<=n;++i) fa[i]=i;
    sort(e+1,e+1+m);
    int tcnt=0,ans=0;
    for(int i=1;i<=m;++i){
        int u=e[i].u,v=e[i].v;
        if(Findfa(u)==Findfa(v)) continue;
        Merge(u,v);
        ans+=e[i].w;
        if(++tcnt==n-1) return ans;
    }
    return -1;
}

```

- *Tarjan*
- 使用时注释掉一部分

```

// vector<int>br; //桥
// bool cut[maxn]; //割点
// int s[maxn],top; //判断scc,edcc,vdcc
// vector<vector<int>> ans; //判断scc,edcc,vdcc
// bool ins[maxn]; //判断scc
int dfn[maxn],low[maxn],tim;

void Tarjan(int u,int in=0){ // in用于判断edcc,vdcc
    //if(!in&&!head[u]) return ans.push_back({u}),void();//特判vdcc
    dfn[u] = low[u] = ++tim;

    //s[++top]=u; //当需要得到edcc,vdcc,scc时
    //ins[u]=1; //当需要scc时
    int child=0;
    for (int i=head[u];i;i=e[i].nx) {
        int v=e[i].v;
        if (!dfn[v]) {
            Tarjan(v,i); //判断edcc,vdcc
            ++child;
            low[u] = min(low[u], low[v]);
            //if(low[v]>dfn[u]) br.push_back(i); //桥

            // if(in&&low[v]>=dfn[u]) cut[u]=1; //割点

            // vdcc
            // if(low[v]>=dfn[u]){
            //     vector<int>vdcc;
            //     while(1){
            //         vdcc.push_back(s[top]);
            //         if(s[top--]==v) break;
            //     }
            //     vdcc.push_back(u);
            //     ans.push_back(vdcc);
            // }
        } else // if(ins[v]) //判断scc // if(i!=(in^1)) //判断edcc,vdcc,桥
            low[u] = min(low[u], dfn[v]);
    }

    // if(child>=2&&(!in)) cut[u]=1; //割点判断

```

```
    //edcc和scc
    // if(dfn[u]==low[u]){
    //     vector<int> secc;
    //     while(1){
    //         secc.push_back(s[top]);
    //         ins[s[top]]=0;    //scc
    //         if(s[top--]==u) break;
    //     }
    //     ans.push_back(secc);
    // }
}
```

- *Dinic*
- 最大流

```

namespace Dinic{
    struct Edge{
        ll v,w,nx;
    }e[maxn];
    int head[maxn],idx=1;
    void AddEdge(int u,int v,int w){
        e[++idx]={v,w,head[u]};
        head[u]=idx;
        e[++idx]={u,0,head[v]};
        head[v]=idx;
    }
    int s,t,cur[maxn],dis[maxn];
    ll Dfs(int u,ll las){
        if(u==t) return las;
        ll res=0;
        for(int i=cur[u];i&&las;i=e[i].nx){
            cur[u]=i;
            ll v=e[i].v,w=min(las,e[i].w);
            if(dis[v]==dis[u]+1&&w){
                ll k=Dfs(v,w);
                res+=k,las-=k;
                e[i].w-=k,e[i^1].w+=k;
            }
        }
        if(!res) dis[u]=INF;
        return res;
    }
    bool Bfs(int s,int t){
        queue<int>q;
        memcpy(cur,head,sizeof(head));
        memset(dis,0x3f,sizeof(dis));
        q.push(s);
        dis[s]=0;
        while(!q.empty()){
            int u=q.front();q.pop();
            for(int i=head[u];i;i=e[i].nx){
                int v=e[i].v,w=e[i].w;
                if(dis[v]==INF&&w)
                    dis[v]=dis[u]+1,q.push(v);
            }
        }
    }
}

```



```

    }
}
return dis[t]!=INF;
}
ll Flow(int S,int T){
    ll ans=0;
    s=S,t=T;
    while(Bfs(s,t))
        ans+=Dfs(s,INF);
    return ans;
}
}

```

数学

- 快速幂

```

int FastPow(int a,int x){
    int res=1;
    while(x){
        if(x&1) res*=a;
        a*=a;
        x>>=1;
    }
    return res;
}

```

- 高精度

```

struct BigInt {
    vector<int> s;
    BigInt(string num = "0") {
        for (int i = num.size() - 1; i >= 0; i--) s.push_back(num[i] - '0');
        trim();
    }
    BigInt(long long num){
        while(num){
            s.push_back(num%10);
            num/=10;
        }
    }
    void trim() {
        while (s.size() > 1 && s.back() == 0) s.pop_back();
    }
    void print() const {
        for (int i = s.size() - 1; i >= 0; i--) cout << s[i];
    }
    BigInt operator+(const BigInt &A)const{
        BigInt C;
        C.s.clear();
        int t = 0;
        for (int i = 0; i < (int)max(A.s.size(), s.size()); i++) {
            if (i < A.s.size()) t += A.s[i];
            if (i < s.size()) t += s[i];
            C.s.push_back(t % 10);
            t /= 10;
        }
        if (t) C.s.push_back(t);
        return C;
    }
    bool operator<(const BigInt &A)const{
        if (A.s.size() != s.size()) return A.s.size() > s.size();
        for (int i = A.s.size() - 1; i >= 0; i--)
            if (A.s[i] != s[i]) return A.s[i] > s[i];
        return true;
    }
    bool operator>(const BigInt &A)const{
        if (A.s.size() != s.size()) return A.s.size() < s.size();
    }

```

```

        for (int i = A.s.size() - 1; i >= 0; i--)
            if (A.s[i] != s[i]) return A.s[i] < s[i];
        return 0;
    }

    bool operator==(const BigInt &A)const{
        if (A.s.size() != s.size()) return 0;
        for(int i=A.s.size()-1;i>=0;--i) if(A.s[i]!=s[i]) return 0;
        return 1;
    }

    bool operator!=(const BigInt &A)const{
        if (A.s.size() != s.size()) return 1;
        for(int i=A.s.size()-1;i>=0;--i) if(A.s[i]!=s[i]) return 1;
        return 0;
    }

    BigInt operator-(const BigInt &B)const{
        BigInt C;
        C.s.clear();
        int t = 0;
        for (int i = 0; i < (int)s.size(); i++) {
            t = s[i] - t;
            if (i < B.s.size()) t -= B.s[i];
            C.s.push_back((t + 10) % 10);
            t = (t < 0);
        }
        C.trim();
        return C;
    }

    BigInt operator*(const BigInt &A)const{
        BigInt C;
        C.s.assign(A.s.size() + s.size(), 0);
        for (int i = 0; i < (int)A.s.size(); i++) {
            int t = 0;
            for (int j = 0; j < (int)s.size() || t; j++) {
                long long cur = C.s[i + j] + 111 * A.s[i] * (j < s.size() ? s[j] : 0) + t;
                C.s[i + j] = cur % 10;
                t = cur / 10;
            }
        }
        C.trim();
        return C;
    }

```

```

    }
    BigInt operator*(const long long &b)const{
        BigInt C;
        C.s.clear();
        long long t = 0;
        for (int i = 0; i < (int)s.size() || t; i++) {
            if (i < s.size()) t += 11l * s[i] * b;
            C.s.push_back(t % 10);
            t /= 10;
        }
        C.trim();
        return C;
    }
    BigInt operator/(const long long &b)const{
        BigInt C;
        C.s.clear();
        long long r = 0;
        for (int i = s.size() - 1; i >= 0; i--) {
            r = r * 10 + s[i];
            C.s.push_back(r / b);
            r %= b;
        }
        reverse(C.s.begin(), C.s.end());
        C.trim();
        return C;
    }
    long long operator%(const long long &b)const{
        long long r = 0;
        for (int i = s.size() - 1; i >= 0; i--) {
            r = r * 10 + s[i];
            r %= b;
        }
        return r;
    }
};

```

- 矩阵

```

struct Matrix{
    int a[maxn][maxn];
    int R,C;
    Matrix(int R=n,int C=n,int x=0):R(R),C(C){
        memset(a,0,sizeof(a));
        for(int i=1;i<=R;++i) a[i][i]=x;
    }
    int* operator[](const int &x){
        return a[x];
    }
    Matrix operator*(Matrix x){
        Matrix res(R,x.C);
        for(int i=1;i<=R;++i) for(int j=1;j<=x.C;++j) for(int k=1;k<=C;++k)
            res[i][j]=(res[i][j]+a[i][k]*x[k][j]);
        return res;
    }
    Matrix operator+(Matrix &x){
        Matrix res(R,C);
        for(int i=1;i<=R;++i) for(int j=1;j<=C;++j) res[i][j]=a[i][j]+x[i][j];
        return res;
    }
    Matrix Pow(int t){
        Matrix A=*this,res(R,C,1);
        while(t){
            if(t&1) res=res*A;
            A=A*A;
            t>>=1;
        }
        return res;
    }
}

// 高斯消元
int Gauss(){
    int r=1,c=1;
    for(;c<=n;++c){
        int cur=r;
        for(int i=r;i<=n;++i) if(fabs(a[i][c])>fabs(a[cur][c])) cur=i;
        if(fabs(a[cur][c])<eps) continue;
        swap(a[cur],a[r]);
        for(int i=n+1;i>=c;--i) a[r][i]/=a[r][c];
    }
}

```

```

        for(int i=r+1;i<=n;++i) if(fabs(a[i][c])>eps) for(int j=n+1;j>=c;--j)
            a[i][j]-=a[r][j]*a[i][c];
        ++r;
    }
    if(r<=n){
        for(int i=r;i<=n;++i) if(fabs(a[i][n+1])>eps) return -1;
        return n-r+1;
    }
    for(int i=n-1;i;i--) for(int j=i+1;j<=n;++j)
        a[i][n+1]-=a[j][n+1]*a[i][j];
    return 0;
}
};

```

- 数论

```

struct NubThe{
    int inv[maxn];
    void GetInv(){
        inv[1]=1;
        for(int i=2;i<=n;++i)
            inv[i]=(mod-mod/i)*inv[mod%i]%mod;
    }
    int ExGcd(int a,int b,int &x,int &y){
        if(!b) return x=1,y=0,a;
        int d=ExGcd(b,a%b,y,x);
        y-=a/b*x;
        return d;
    }
    vector<int>p;
    int phi[maxn];
    bool nop[maxn];
    void GetP(){
        phi[1]=1;
        for(int i=2;i<=n;++i){
            if(!nop[i]) p.push_back(i),phi[i]=i-1;
            for(int j:p){
                nop[i*j]=1;
                if(!(i%j)){
                    phi[i*j]=phi[i]*j;
                    break;
                }else phi[i*j]=phi[i]*phi[j];
            }
        }
    }
};

```

- 组合数学

```

struct Combin{
    ll Catalan[maxn];
    ll fac[maxn];
    void GetCatalan(){
        Catalan[1]=1;
        for(int i=2;i<=n;++i)
            Catalan[i]=Catalan[i-1]*(4*i-2)/(i+1);
    }
    // ll C(int a,int b){
    //     return fac[a]/(fac[b]*fac[a-b]);
    // }
    void GetFac(){
        fac[0]=1;
        for(int i=1;i<=n;++i)
            fac[i]=fac[i-1]*i;
    }
    ll C[maxn][maxn],sum[maxn][maxn];
    void GetC(){
        C[0][0]=sum[0][0]=1;
        for(int i=1;i<=maxn-1;++i){
            C[i][i]=C[i][0]=sum[i][0]=1;
            for(int j=1;j<=maxn-1;++j)
                C[i][j]=(C[i-1][j-1]+C[i-1][j])%mod;
            for(int j=1;j<=maxn-1;++j)
                sum[i][j]=(sum[i][j-1]+C[i][j])%mod;
        }
    }
};

```

计算几何

- AI写的。不过应该没锅。


```

const double eps = 1e-9;
const double Pi = acos(-1.0);

// 比较函数
inline int dcmp(double x) {
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}

// 点/向量结构体
struct Point {
    double x, y;

    Point() : x(0), y(0) {}
    Point(double _x, double _y) : x(_x), y(_y) {}

    // 输入输出
    friend istream& operator>>(istream& is, Point& p) {
        return is >> p.x >> p.y;
    }
    friend ostream& operator<<(ostream& os, const Point& p) {
        return os << "(" << p.x << ", " << p.y << ")";
    }

    // 比较运算
    bool operator==(const Point& p) const {
        return dcmp(x - p.x) == 0 && dcmp(y - p.y) == 0;
    }
    bool operator!=(const Point& p) const {
        return !(*this == p);
    }
    bool operator<(const Point& p) const {
        return dcmp(x - p.x) < 0 || (dcmp(x - p.x) == 0 && dcmp(y - p.y) < 0);
    }

    // 向量运算
    Point operator+(const Point& p) const {
        return Point(x + p.x, y + p.y);
    }
}

```

```

Point operator-(const Point& p) const {
    return Point(x - p.x, y - p.y);
}

Point operator*(double k) const {
    return Point(x * k, y * k);
}

Point operator/(double k) const {
    return Point(x / k, y / k);
}

// 向量点积
double dot(const Point& p) const {
    return x * p.x + y * p.y;
}

// 向量叉积
double cross(const Point& p) const {
    return x * p.y - y * p.x;
}

// 向量长度
double length() const {
    return sqrt(x * x + y * y);
}

// 向量长度的平方
double length2() const {
    return x * x + y * y;
}

// 向量旋转（逆时针旋转rad弧度）
Point rotate(double rad) const {
    return Point(x * cos(rad) - y * sin(rad),
                x * sin(rad) + y * cos(rad));
}

// 向量旋转90度（逆时针）
Point rotate90() const {
    return Point(-y, x);
}

```

```

// 单位向量
Point unit() const {
    double len = length();
    return Point(x / len, y / len);
}

// 规范化向量（保持方向不变，长度为1）
void normalize() {
    double len = length();
    x /= len;
    y /= len;
}

// 向量夹角（弧度）
double angle() const {
    return atan2(y, x);
}

// 与另一向量的夹角（弧度）
double angle(const Point& p) const {
    return acos(dot(p) / (length() * p.length()));
}

// 判断向量是否为零向量
bool isZero() const {
    return dcmp(x) == 0 && dcmp(y) == 0;
}
};

// 直线/线段结构体
struct Line {
    Point s, e; // 起点和终点

    Line() {}
    Line(Point _s, Point _e) : s(_s), e(_e) {}

    // 方向向量
    Point direction() const {
        return e - s;
    }
};

```

```

}

// 直线长度
double length() const {
    return (e - s).length();
}

// 点到直线的距离
double distanceToPoint(const Point& p) const {
    return fabs((p - s).cross(direction())) / direction().length();
}

// 点到直线的垂足
Point projection(const Point& p) const {
    Point v = direction();
    return s + v * ((p - s).dot(v) / v.length2());
}

// 点关于直线的对称点
Point reflection(const Point& p) const {
    Point proj = projection(p);
    return proj * 2 - p;
}

// 判断点是否在线段上
bool onSegment(const Point& p) const {
    // 先判断点是否在直线端点的包围盒内
    if (dcmp((p - s).cross(e - s)) != 0) return false;

    // 再判断点是否在线段上
    return dcmp((p - s).dot(p - e)) <= 0;
}

// 判断点是否在射线上
bool onRay(const Point& p) const {
    if (dcmp((p - s).cross(e - s)) != 0) return false;
    return dcmp((p - s).dot(e - s)) >= 0;
}
};

```

```

// 多边形结构体
struct Polygon {
    vector<Point> points;

    Polygon() {}
    Polygon(const vector<Point>& pts) : points(pts) {}

    // 计算多边形面积（带方向）
    double area() const {
        double res = 0;
        int n = points.size();
        for (int i = 0; i < n; i++) {
            int j = (i + 1) % n;
            res += points[i].cross(points[j]);
        }
        return res / 2.0;
    }

    // 计算多边形面积（绝对值）
    double absArea() const {
        return fabs(area());
    }

    // 计算多边形周长
    double perimeter() const {
        double res = 0;
        int n = points.size();
        for (int i = 0; i < n; i++) {
            int j = (i + 1) % n;
            res += (points[j] - points[i]).length();
        }
        return res;
    }

    // 判断点是否在多边形内（射线法）
    bool contains(const Point& p) const {
        int n = points.size();
        int cnt = 0;

        for (int i = 0; i < n; i++) {

```

```

    int j = (i + 1) % n;
    Point a = points[i], b = points[j];

    // 如果点在顶点上
    if (p == a || p == b) return true;

    // 如果点在边上
    if (Line(a, b).onSegment(p)) return true;

    // 射线与边相交判断
    if (dcmp(a.y - b.y) > 0) swap(a, b);
    if (dcmp(p.y - a.y) > 0 && dcmp(p.y - b.y) <= 0 &&
        dcmp((b - a).cross(p - a)) > 0) {
        cnt++;
    }
}

return cnt % 2 == 1;
}
};

// 几何工具函数
struct Geometry {
    // 判断三点共线
    static bool collinear(const Point& a, const Point& b, const Point& c) {
        return dcmp((b - a).cross(c - a)) == 0;
    }

    // 判断点是否在线段上
    static bool pointOnSegment(const Point& p, const Point& a, const Point& b) {
        if (collinear(p, a, b)) {
            return dcmp((p - a).dot(p - b)) <= 0;
        }
        return false;
    }

    // 判断两线段是否相交（包括端点）
    static bool segmentIntersect(const Point& a, const Point& b,
                                  const Point& c, const Point& d) {
        if (max(a.x, b.x) < min(c.x, d.x) ||

```

```

        max(a.y, b.y) < min(c.y, d.y) ||
        min(a.x, b.x) > max(c.x, d.x) ||
        min(a.y, b.y) > max(c.y, d.y)) {
            return false;
        }

        double cross1 = (b - a).cross(c - a);
        double cross2 = (b - a).cross(d - a);
        double cross3 = (d - c).cross(a - c);
        double cross4 = (d - c).cross(b - c);

        return dcmp(cross1) * dcmp(cross2) <= 0 &&
            dcmp(cross3) * dcmp(cross4) <= 0;
    }

    // 判断两直线是否平行
    static bool parallel(const Point& a, const Point& b,
                        const Point& c, const Point& d) {
        return dcmp((b - a).cross(d - c)) == 0;
    }

    // 判断两直线是否垂直
    static bool perpendicular(const Point& a, const Point& b,
                             const Point& c, const Point& d) {
        return dcmp((b - a).dot(d - c)) == 0;
    }

    // 计算两直线交点（确保不平行）
    static Point lineIntersection(const Point& a, const Point& b,
                                 const Point& c, const Point& d) {
        Point v1 = b - a, v2 = d - c;
        double cross = v1.cross(v2);
        if (dcmp(cross) == 0) return Point(NAN, NAN); // 平行

        Point u = c - a;
        double t = v2.cross(u) / cross;
        return a + v1 * t;
    }

    // 计算点到线段的最短距离

```

```

static double pointToSegmentDistance(const Point& p,
                                     const Point& a, const Point& b) {
    if (a == b) return (p - a).length();

    Point v = b - a, w = p - a;
    double proj = w.dot(v);

    if (proj <= 0) return (p - a).length();
    if (proj >= v.length2()) return (p - b).length();

    return fabs(v.cross(w)) / v.length();
}

// 计算两线段的最短距离
static double segmentToSegmentDistance(const Point& a, const Point& b,
                                       const Point& c, const Point& d) {
    if (segmentIntersect(a, b, c, d)) return 0.0;

    double d1 = min(pointToSegmentDistance(a, c, d),
                    pointToSegmentDistance(b, c, d));
    double d2 = min(pointToSegmentDistance(c, a, b),
                    pointToSegmentDistance(d, a, b));

    return min(d1, d2);
}

// 计算三角形的面积
static double triangleArea(const Point& a, const Point& b, const Point& c) {
    return fabs((b - a).cross(c - a)) / 2.0;
}

// 判断点是否在三角形内（包括边界）
static bool pointInTriangle(const Point& p, const Point& a,
                            const Point& b, const Point& c) {
    double s1 = triangleArea(p, a, b);
    double s2 = triangleArea(p, b, c);
    double s3 = triangleArea(p, c, a);
    double s = triangleArea(a, b, c);

    return dcmp(s1 + s2 + s3 - s) == 0;
}

```



```

}

// 计算多边形的重心
static Point polygonCentroid(const vector<Point>& points) {
    double area = 0;
    Point centroid(0, 0);
    int n = points.size();

    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        double cross = points[i].cross(points[j]);
        area += cross;
        centroid.x += (points[i].x + points[j].x) * cross;
        centroid.y += (points[i].y + points[j].y) * cross;
    }

    area /= 2.0;
    centroid.x /= (6.0 * area);
    centroid.y /= (6.0 * area);

    return centroid;
}

// 计算两点间的距离
static double distance(const Point& a, const Point& b) {
    return (b - a).length();
}

// 计算两点间距离的平方
static double distance2(const Point& a, const Point& b) {
    return (b - a).length2();
}

// 计算向量的极角（角度制）
static double angleDegree(const Point& p) {
    double angle = atan2(p.y, p.x) * 180.0 / Pi;
    if (angle < 0) angle += 360.0;
    return angle;
}

```

```

// 判断点是否在凸多边形内（二分法， $O(\log n)$ ）
static bool pointInConvexPolygon(const Point& p, const vector<Point>& poly) {
    int n = poly.size();

    // 判断点在第一条边和最后一条边的位置
    if (dcmp((poly[1] - poly[0]).cross(p - poly[0])) < 0 ||
        dcmp((poly[n-1] - poly[0]).cross(p - poly[0])) > 0) {
        return false;
    }

    // 二分查找
    int l = 1, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        if (dcmp((poly[mid] - poly[0]).cross(p - poly[0])) > 0) {
            l = mid;
        } else {
            r = mid;
        }
    }

    // 判断点是否在三角形内
    return pointInTriangle(p, poly[0], poly[l], poly[r]);
}

// 获取凸包（Graham扫描算法）
static Polygon convexHull(vector<Point> points) {
    int n = points.size();
    if (n <= 3) return Polygon(points);

    // 寻找最下最左的点
    int base = 0;
    for (int i = 1; i < n; i++) {
        if (dcmp(points[i].y - points[base].y) < 0 ||
            (dcmp(points[i].y - points[base].y) == 0 &&
             dcmp(points[i].x - points[base].x) < 0)) {
            base = i;
        }
    }
    swap(points[0], points[base]);

```

```

// 按极角排序
sort(points.begin() + 1, points.end(), [&](const Point& a, const Point& b) {
    double cross = (a - points[0]).cross(b - points[0]);
    if (dcmp(cross) != 0) return cross > 0;
    return (a - points[0]).length2() < (b - points[0]).length2();
});

// Graham扫描
vector<Point> hull;

for (int i = 0; i < n; i++) {
    while (hull.size() >= 2) {
        Point a = hull[hull.size() - 2];
        Point b = hull.back();
        Point c = points[i];

        if (dcmp((b - a).cross(c - a)) <= 0) {
            hull.pop_back();
        } else {
            break;
        }
    }
    hull.push_back(points[i]);
}

return Polygon(hull);
}
};

// 圆结构体
struct Circle {
    Point center;
    double radius;

    Circle() : radius(0) {}
    Circle(Point c, double r) : center(c), radius(r) {}

    // 判断点是否在圆内（包括边界）
    bool contains(const Point& p) const {

```

```

        return dcmp((p - center).length() - radius) <= 0;
    }

    // 计算圆的面积
    double area() const {
        return Pi * radius * radius;
    }

    // 计算圆的周长
    double perimeter() const {
        return 2 * Pi * radius;
    }

    // 获取圆与直线的交点
    vector<Point> intersectLine(const Line& line) const {
        vector<Point> res;
        Point proj = line.projection(center);
        double dist = (proj - center).length();

        if (dcmp(dist - radius) > 0) {
            return res; // 无交点
        }

        if (dcmp(dist - radius) == 0) {
            res.push_back(proj); // 相切
            return res;
        }

        // 两个交点
        double d = sqrt(radius * radius - dist * dist);
        Point dir = line.direction().unit();
        res.push_back(proj + dir * d);
        res.push_back(proj - dir * d);

        return res;
    }

    // 获取两圆的交点
    vector<Point> intersectCircle(const Circle& other) const {
        vector<Point> res;

```

```

Point c1 = center, c2 = other.center;
double r1 = radius, r2 = other.radius;

double d = (c2 - c1).length();
if (dcmp(d) == 0) {
    return res; // 同心圆
}

if (dcmp(d - r1 - r2) > 0 || dcmp(fabs(r1 - r2) - d) > 0) {
    return res; // 无交点
}

double a = (r1 * r1 - r2 * r2 + d * d) / (2 * d);
double h = sqrt(r1 * r1 - a * a);

Point p = c1 + (c2 - c1) * (a / d);

if (dcmp(h) == 0) {
    res.push_back(p); // 相切
    return res;
}

Point offset = (c2 - c1).rotate90().unit() * h;
res.push_back(p + offset);
res.push_back(p - offset);

return res;
}

```

// 获取过圆外一点的切线

```

vector<Line> tangentsFromPoint(const Point& p) const {
    vector<Line> res;

    double d = (p - center).length();
    if (dcmp(d - radius) < 0) {
        return res; // 点在圆内
    }

    if (dcmp(d - radius) == 0) {

```

```
    // 点在圆上，只有一条切线
    Point dir = (p - center).rotate90();
    res.push_back(Line(p, p + dir));
    return res;
}

// 两条切线
double angle = asin(radius / d);
Point dir = (center - p).unit();

Point p1 = p + dir.rotate(angle) * d;
Point p2 = p + dir.rotate(-angle) * d;

res.push_back(Line(p, p1));
res.push_back(Line(p, p2));

return res;
}
};
```

字符串

- *KMP*

```

struct KMP{
    int fal[maxn];
    void GetFail(char *b){
        int j=0;
        for(int i=2;i<=m;++i){
            while(j&&b[j+1]!=b[i]) j=fal[j];
            if(b[j+1]==b[i]) ++j;
            fal[i]=j;
        }
    }
}

vector<int> kmp(char *a,char *b){
    int j=0;
    GetFail(b);
    vector<int> ans;
    for(int i=1;i<=n;++i){
        while (j&&b[j+1]!=a[i])j=fal[j];
        if(b[j+1]==a[i]) ++j;
        if(j==m) ans.push_back(i-j+1),j=fal[j];
    }
    return ans;
}

};

```

- *ACAM*, AC自动机

```

struct ACAM{
    struct Node{
        int ch[26],fal,cnt;
    }t[maxn];
#define ch(p) t[p].ch
#define fal(p) t[p].fal
#define cnt(p) t[p].cnt
    vector<int>e[maxn];
    int pos[maxn],idx;
    void Insert(const string &s,int id){
        int now=0;
        for(char ch:s){
            if(!ch(now)[ch-'a'])
                ch(now)[ch-'a']=++idx;
            now=ch(now)[ch-'a'];
        }
        pos[id]=now;
    }
    void GetFail(){
        queue<int>q;
        for(int i=0;i<26;++i) if(ch(0)[i])
            q.push(ch(0)[i]);
        while(q.size()){
            int u=q.front();q.pop();
            for(int i=0;i<26;++i){
                int v=ch(u)[i];
                if(v) fal(v)=ch(fal(u))[i],q.push(v);
                else ch(u)[i]=ch(fal(u))[i];
            }
        }
        for(int i=1;i<=idx;++i)
            e[fal(i)].push_back(i);
    }
    void Dfs(int u){
        for(int v:e[u])
            Dfs(v),cnt(u)+=cnt(v);
    }
    vector<int> Query(const string &s){
        int now=0;

```



```

        for(char ch:s)
            now=ch(now)[ch-'a'],cnt(now)++;
        Dfs(0);
        vector<int>ans;
        for(int i=1;i<=n;++i)
            ans.push_back(cnt(pos[i]));
        return ans;
    }
};

```

杂项

- 快读

```

char buf[maxn<<4],*_s,*_t;
#define gc() (_s==_t && (_t=(_s=buf)+fread(buf,1,maxn<<4,stdin),_s==_t)?EOF:*_s++)
void read(ll &x){
    int f=1;x=0;
    char ch=gc();
    for(;!isdigit(ch);ch=gc())
        if(ch=='-') f=-1;
    for(;isdigit(ch);ch=gc())
        x=x*10+ch-'0';
    x*=f;
}

```